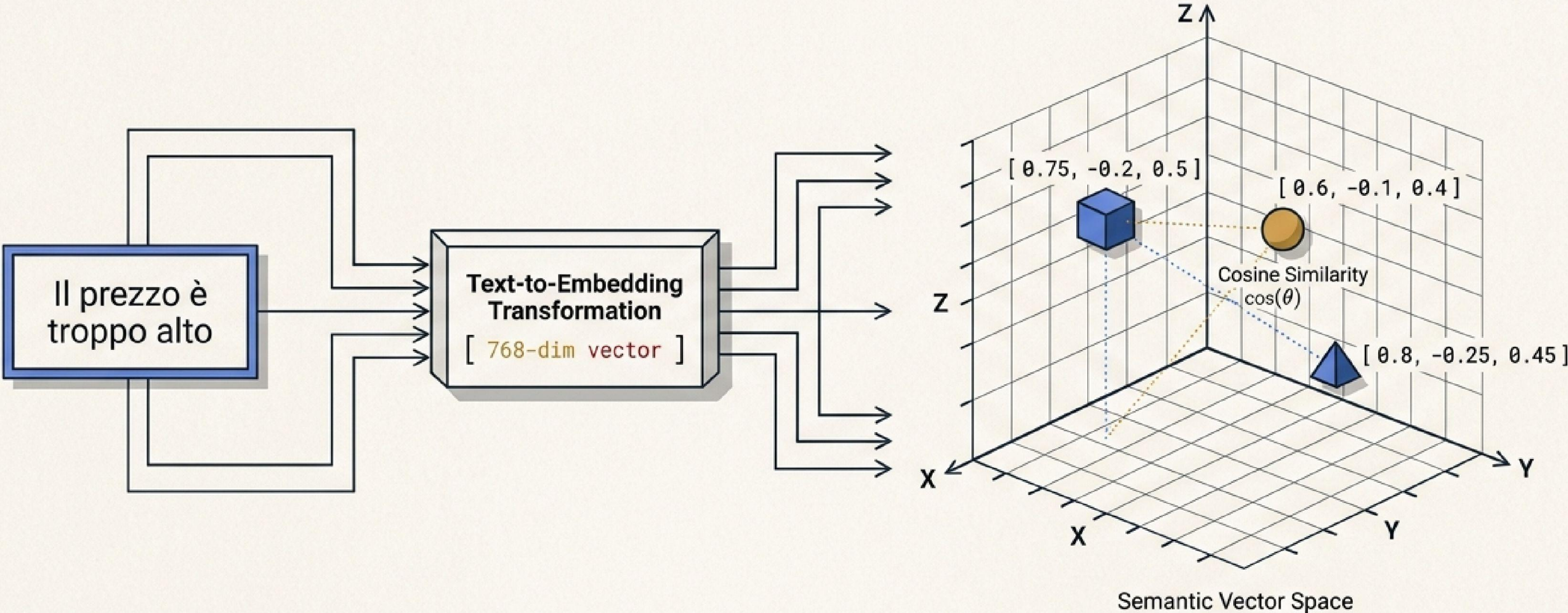


Embeddings & Ricerca Semantica

Trasformare il testo in numeri per cercare per significato.

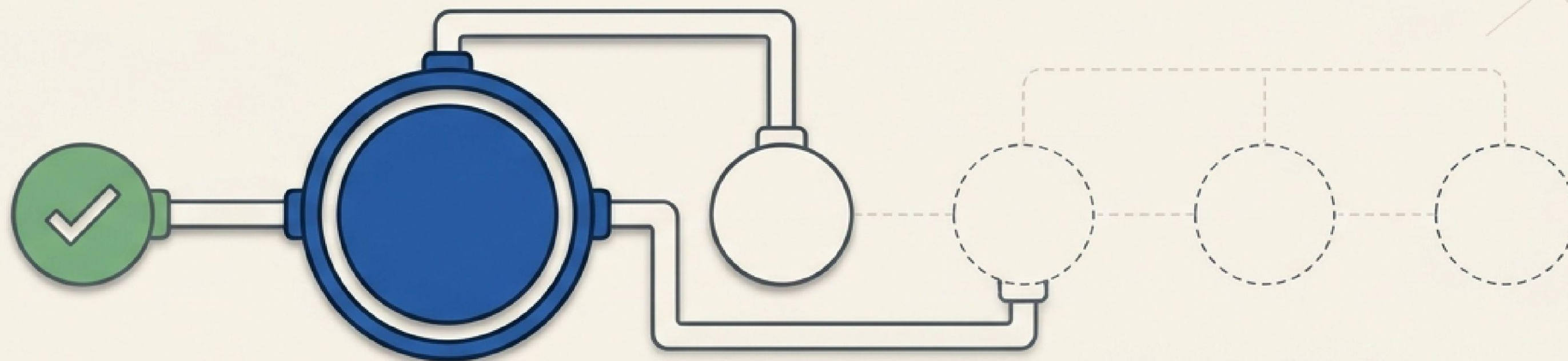
Corso NLP & Generative AI — Lezione 2 di 6. Ambiente: Google Colab (GPU T4).

Model: BERT-base-italian
Dimensions: 768
Metric: Cosine
Latency: 25ms



Model: BERT-base-italian	Dimensions: 768	[embedding = [0.12, 0.45, -0.3, ..., 0.08] # 768 dims]
Metric: Cosine	Latency: 25ms	

La pipeline NLP: dove siamo arrivati



Lezione 1:
Setup & Sentiment
Zero-Shot

Lezione 2:
Embeddings
(Oggi)



Il contesto rimane lo stesso: **200 recensioni clienti sintetiche** in italiano. **Stesso seed riproducibile**, nuovo mattone tecnologico.

Execution Plan: Obiettivi di oggi

[] Decodificare gli embeddings e la struttura dello spazio latente

[] Misurare le distanze vettoriali tramite la similarità coseno

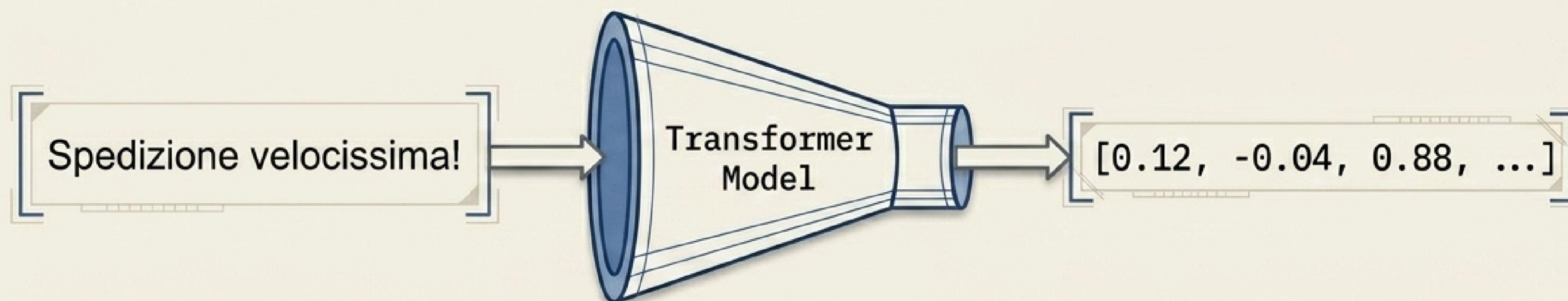
[] Eseguire il batch-encoding su tutte le 200 recensioni

[] Architetture un motore di ricerca semantica da zero

[] (Bonus) Renderizzare la topologia 2D tramite PCA

Dal testo al tensore: la meccanica dell'embedding

Un embedding riduce un testo di lunghezza variabile a un vettore a dimensione fissa (qui, 768 valori continui).



- Significato simile -> Vettori numericamente vicini.



- Significato diverso -> Vettori numericamente distanti.



Lo spazio latente: dove abitano i concetti

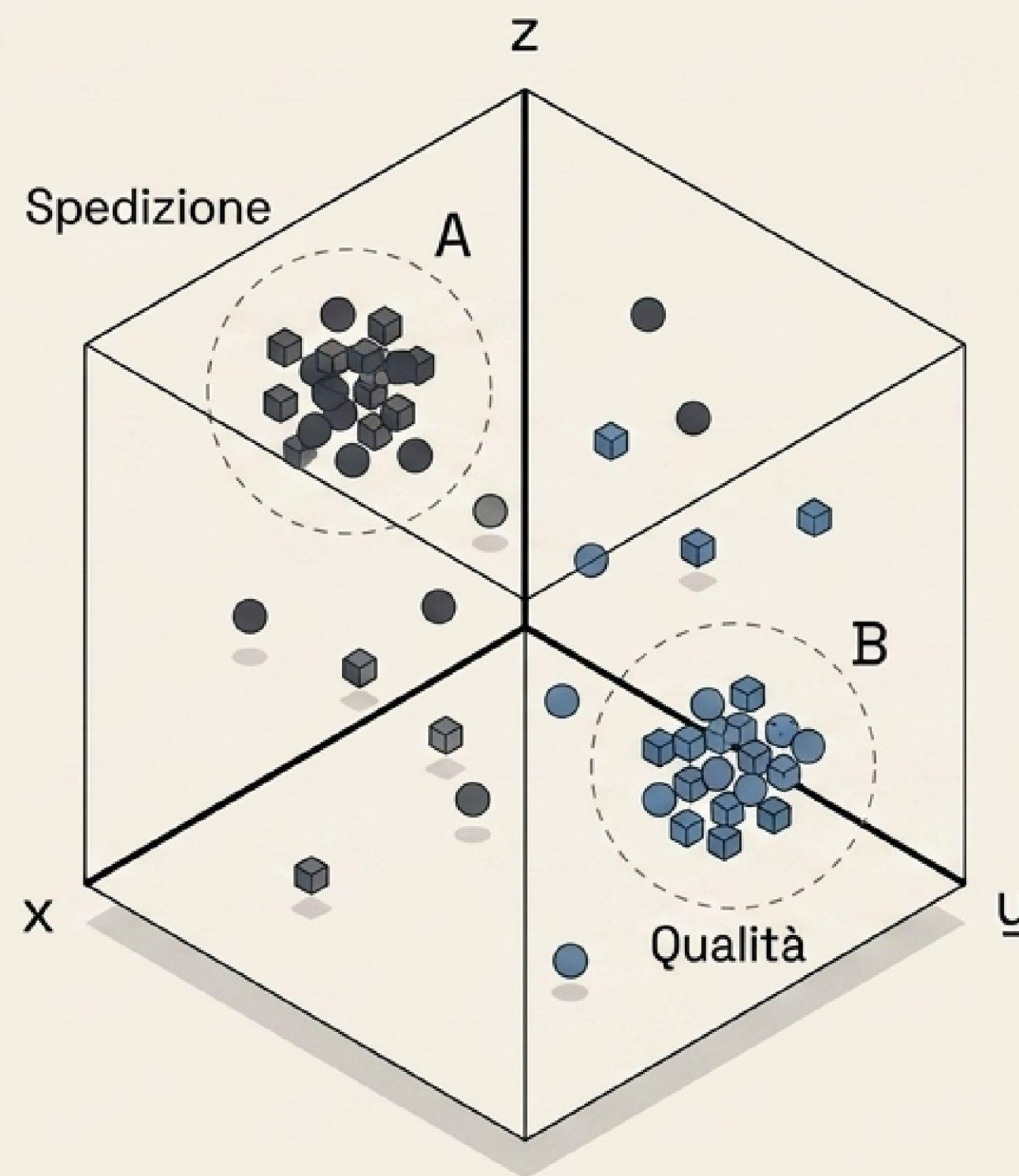
I vettori generati "vivono" in uno spazio a centinaia di dimensioni. Sebbene impossibile da visualizzare per intero, possiamo misurarne la topologia.

Concetti affini collassano in cluster contigui.

Esplora la geometria in tempo reale:
projector.tensorflow.org

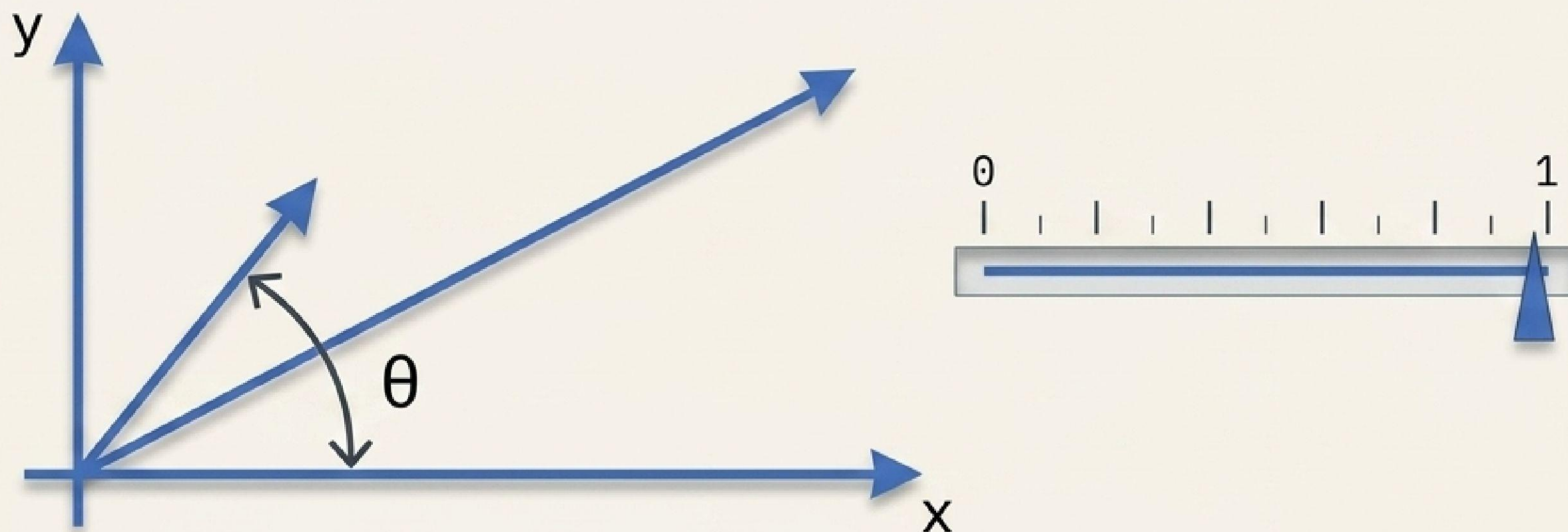
40%

60%



Misurare la prossimità: la similarità coseno

Per calcolare la distanza semantica, non guardiamo la lunghezza del vettore, ma la sua direzione. Usiamo l'angolo theta.



[~ 1 : Direzioni allineate (Significato identico)]
[~ 0 : Vettori ortogonali (Nessuna relazione semantica)]

Optimization: Con vettori normalizzati (lunghezza 1), il calcolo del coseno si riduce a un rapidissimo prodotto scalare.

Hardware & Model Specs

`intfloat/multilingual-e5-base`

Libreria	sentence-transformers
Lingue	Multilingue (Ottimizzato per l'Italiano)
Footprint	Leggero (Esecuzione nativa su GPU T4)
Output	Tensore a 768 dimensioni

```
modello = SentenceTransformer('intfloat/multilingual-e5-base')
```


Syntax Rules: Il requisito dei prefissi E5

La famiglia di modelli E5 utilizza un prompt invisibile per orientare lo spazio latente. Ometterlo degrada silenziosamente la qualità della ricerca.

Visual Diff

Senza Prefissi (Degradato)	Con Prefissi (Corretto)
<pre>[encode('Il pacco è arrivato rotto') encode('problemi di spedizione')]</pre>	<pre>[encode('passage: Il pacco è arrivato rotto') -> Per il database encode('query: problemi di spedizione') -> Per la ricerca utente]</pre>

Generazione del Database Vettoriale

```
embeddings = modello.encode(recensioni, normalize_embeddings=True)
```

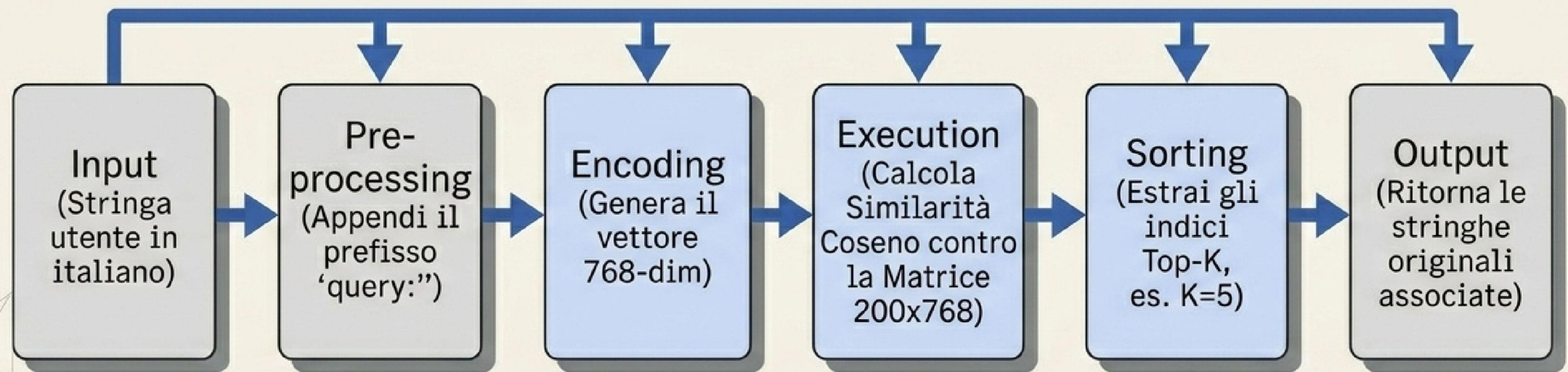
	dim_0	dim_1	dim_2	...	dim_767
0	0.014	-0.821	0.334	...	
1	0.010	-0.493	0.320	...	
2	0.013	-0.275	0.327	...	
...				...	
198				...	
199				...	

Input: Array di 200 stringhe prefissate con passage:

Output: Una singola matrice di shape (200, 768)

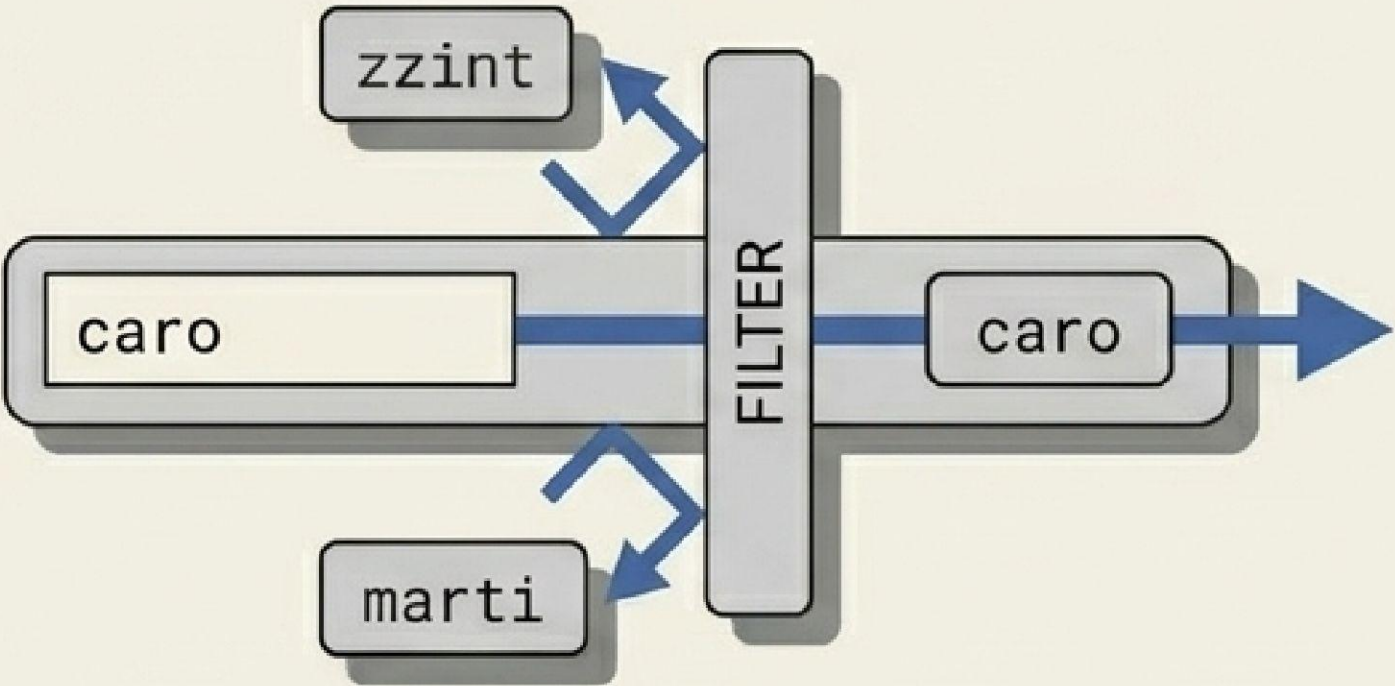
Flag:
normalize_embeddings=True
pre-calcola la lunghezza a 1,
abilitando il calcolo ultra-veloce.

Architettura della funzione cerca()



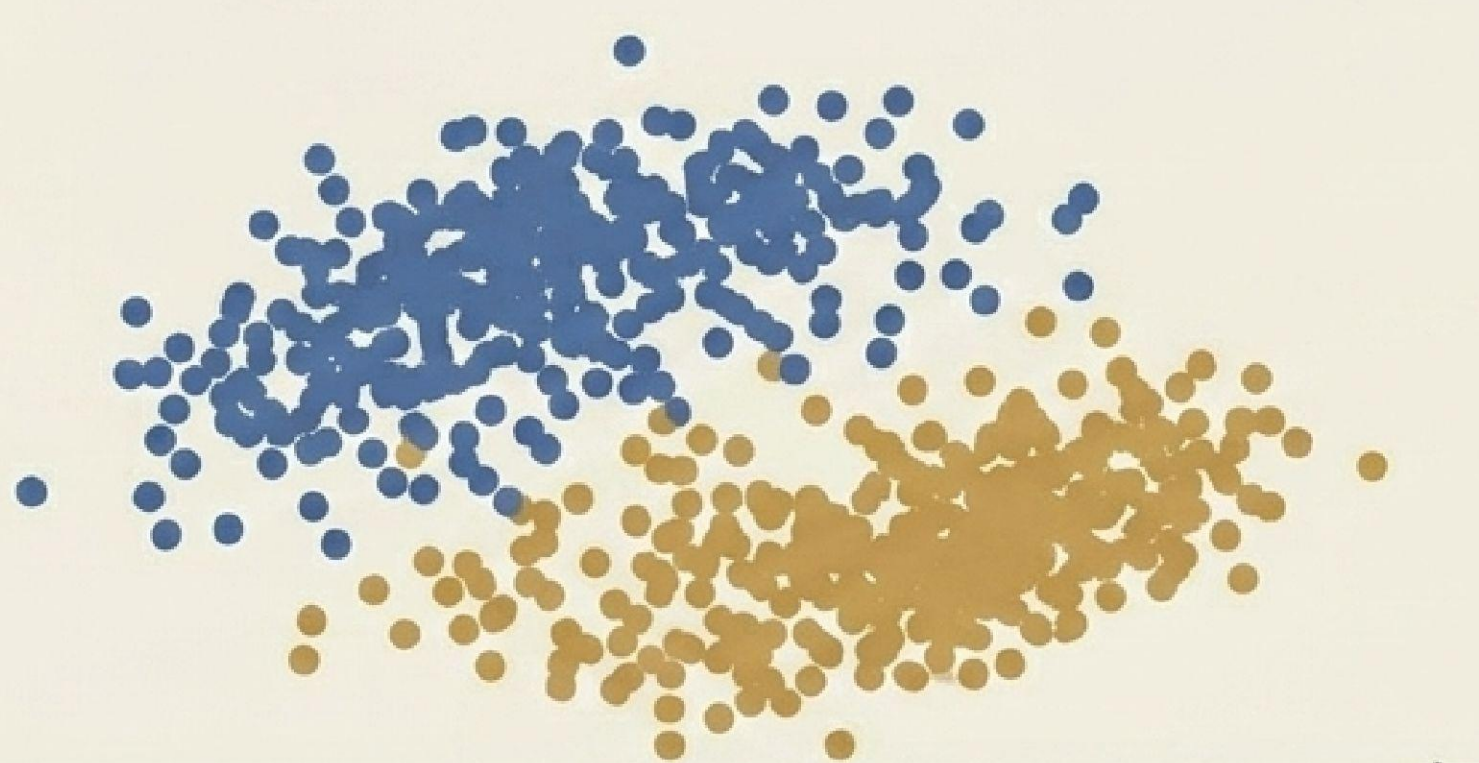
Cambio di paradigma: Boolean vs Vettori

Esempio utente: il prezzo è troppo alto

Ricerca Parola Chiave	Ricerca Semantica
contains('caro')  The diagram illustrates a keyword search process. A horizontal bar contains the word 'caro'. Above this bar, the text 'contains('caro')' is written. To the left of the bar, there are two boxes labeled 'zzint' and 'marti'. Arrows point from these boxes to a vertical bar labeled 'FILTER'. An arrow points from the 'FILTER' bar to the right, passing through the 'caro' box, and then continues to the right as a blue arrow. Trova solo occorrenze esatte. Completamente cieca a sinonimi e parafrasi.	Vettori E5  The diagram illustrates a semantic search process. It shows a central node 'il prezzo è troppo alto' connected by blue lines to three other nodes: 'caro', 'costoso', and 'non giustificato'. Capisce l'intento. Recupera recensioni pertinenti anche se non condividono alcuna parola con la query.

Esecuzione in ambiente Colab

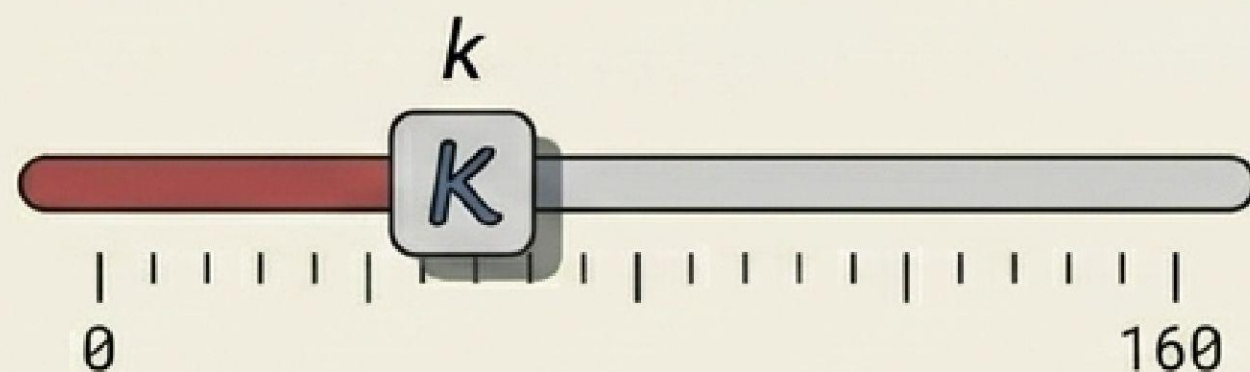
```
> cerca('problemi con la spedizione e i tempi di consegna', k=5)  
[1] passage: Il pacco ha impiegato tre settimane ad arrivare...  
[2] passage: Corriere pessimo, scatola distrutta e ritardo...  
[3] passage: Consegna non rispettata rispetto alle stime...
```



Bonus: Proiezione PCA in 2D. La riduzione dimensionale da 768 a 2 assi ci permette di visualizzare i raggruppamenti latenti basati sul rating originale.

Sfida di implementazione

A) Tuning dei Parametri



```
> query = "..."  
> cerca(query, k=5)
```

Modifica la variabile query e altera il valore di k nella funzione cerca(...).
Analizza come degrada la pertinenza all'aumentare di K.

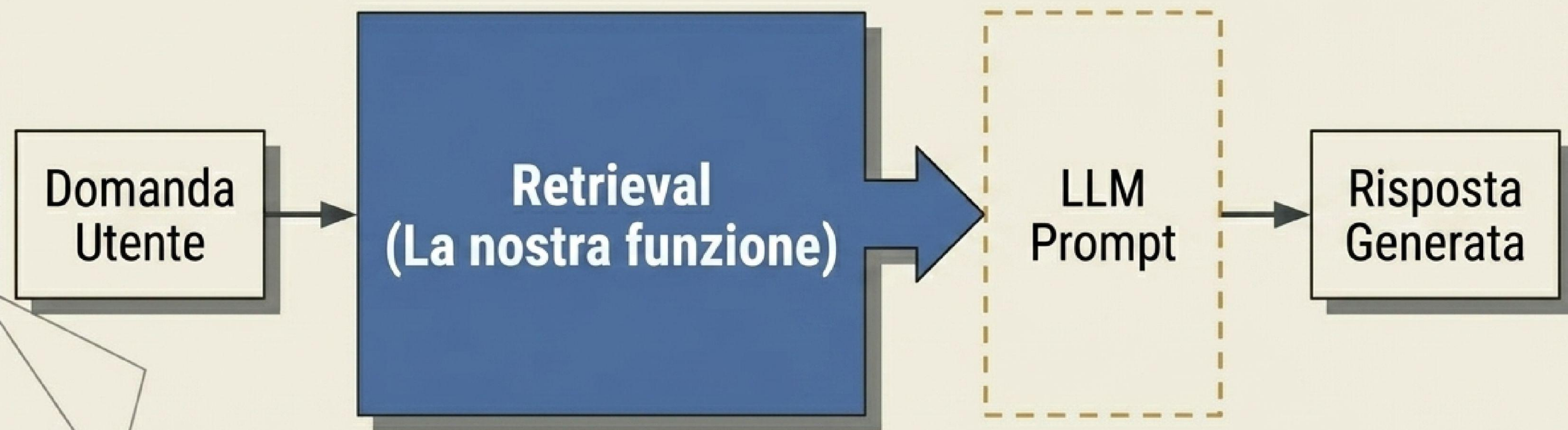
B) Similarità Recensione-Recensione

```
def simili_a(indice, k):  
    [ ]
```

Implementa una nuova funzione `simili_a(indice, k)`.
Input: l'indice di una recensione.
Output: le K recensioni più vicine nello spazio latente (assicurandoti di escludere la recensione sorgente dai risultati!).

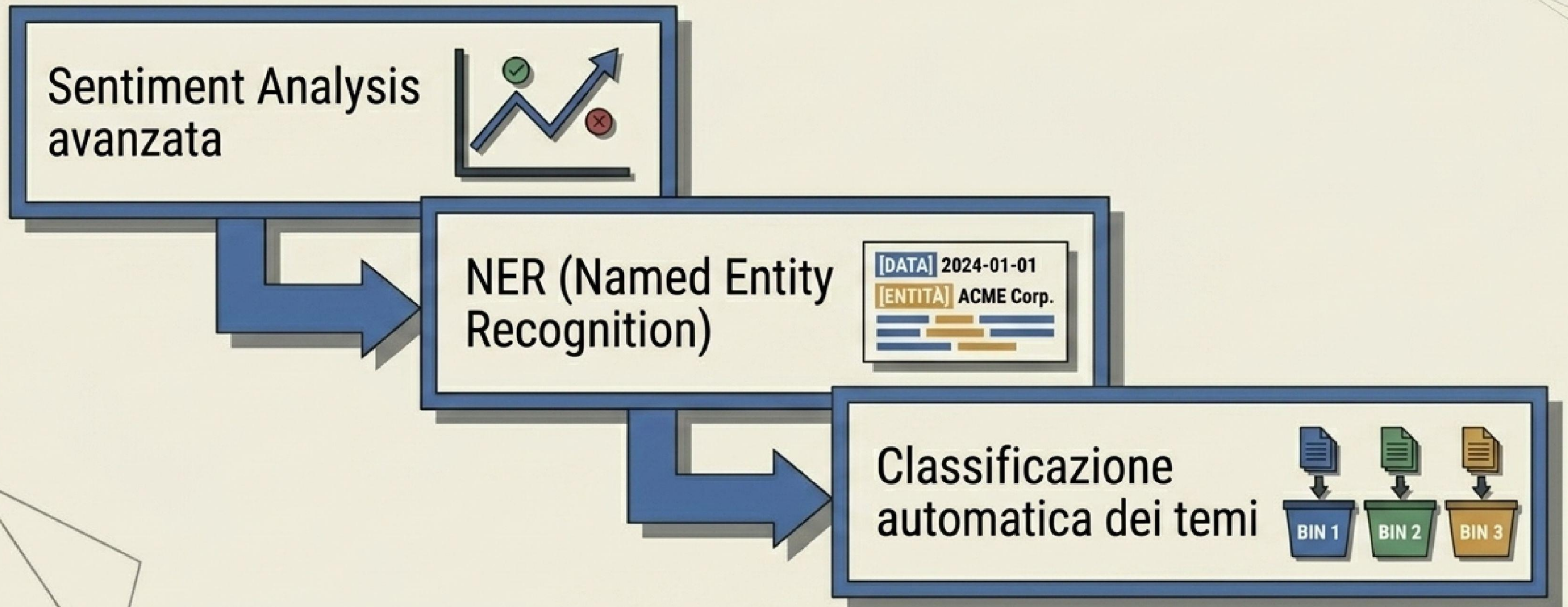
La sintesi: Il motore di un sistema RAG

Il motore di ricerca semantica costruito oggi non è fine a se stesso. La nostra **funzione** `cerca()` è l'esatto componente di **Retrieval all'interno** di un'architettura RAG industriale. Sarà il cuore pulsante che fornirà il **contesto all'applicazione LLM finale** (Lezione 6).



Prossima tappa: Modelli Task-Specific

Nella **Lezione 3** abbandoneremo la pura ricerca per estrarre insight strutturati dal testo.



Repository ufficiale: github.com/ccasadei-maggioli/corso-nlp-genai-2026